

Ch12 练习：存储类别、链接和内存管理

重点章 · 5 题：填空 1 → 改错 1 → 编程 2 → 专业场景拓展 1 + 复习题索引（索引与参考答案区不计题数）。改错题诊断均为本机 gcc -Wall -Wextra 16.1.0 实测。

范围自律：不考多文件工程的链接细节。编译命令：gcc -Wall -Wextra -o prog.exe prog.c

12-1 填空 (★)

填空 ★

补全判断，使描述正确：

函数内的普通局部变量属于____作用域、____存储期。

加 `static` 后，它变成____存储期，____（每次/只一次）初始化。

在函数外声明的变量是____作用域（全局）。

`malloc` 申请的内存存在____（堆/栈）上，用完要____。

12-2 改错 (★★，先自己找再对答案)

改错 ★★

下面程序有 **4 处问题**，找出并改正，分类为「编译期报错 / 仅警告 / 静默逻辑错」：

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int n = 5, i;
    static int x = n;           /* ① */
    int *arr = malloc(n);       /* ② */
    int *p = malloc(10 * sizeof(int)); /* ③ */
    for (i = 0; i < n; i++) arr[i] = i * 2;
    for (i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("x=%d\n", x);
    return 0;                  /* ④ */
}
```

提示：static 初始化时机 / 忘记 sizeof / 申请了不用 / 申请了不还。

12-3 编程 (★★)

编程 ★★

写程序：定义函数 `int next_frame(void)`，用 `static` 计数器从 1 开始递增，返回帧号。`main` 里连续调用 5 次并打印。

输出样例：

```
帧号:1 帧号:2 帧号:3 帧号:4 帧号:5
```

12-4 编程 (★★★)

编程 ★★★

写程序：先读入 `n`，用 `malloc` 申请 `n` 个 `int`，读入 `n` 个数，打印它们的平均值（1 位小数）和最大值，最后 `free`。

输出样例：

```
n=? 5
5 个数: 1 2 3 4 5
平均 = 3.0 最大 = 5
```

12-5 专业场景拓展 (★★★，不需要硬件知识)

拓展 ★★★

内存管理场景：串口缓冲区按需分配。写程序：读入要缓存多少字节 `n`，用 `malloc` 申请 `n` 字节，把字符 'A' 填满并打印前 3 个，最后 `free`（这题练 `malloc/free` 配对 + 指针使用，不需要硬件知识）。

输出样例：

```
缓存大小 n=? 10
缓冲前 3 字节: A A A
```

12-6 复习题索引

打开《C Primer Plus（第6版）》第 12 章，完成复习题 1、3、5。题号对应主教材，原题不入本册。

做完用「把程序运行演一遍」的方式自测 static 行为。

12-7 参考答案（先自己写，再看思路与常见错法）

12-1

依次填：**块、自动、静态、只一次、文件、堆、free。**

思路：作用域回答「哪能用」，存储期回答「活多久」；static 改存储期不改作用域（块内 static 仍只在本块可见）。

常见错法：把「static 让变量变全局」记错——它只是活得久，可见范围不变。

12-2

四处问题及实测诊断（gcc 16.1.0 -Wall -Wextra 实测）：

- ① `static int x = n;` 编译期报错：initializer element is not constant
（static 初始化必须是编译期常量，不能是运行时变量 n）
- ② `malloc(n)` 忘 `sizeof` 静默逻辑错：无诊断。只分到 n 字节却当 n 个 int 用，
堆写越界（后果隐蔽）
- ③ `int *p = malloc(...);` 不用 仅警告：unused variable 'p' [-Wunused-variable]
（顺带：这 10 个 int 也没 free → 泄漏）
- ④ 全程没 `free` 静默逻辑错：无诊断。arr 和 p 都泄漏，
程序退出前内存一直在涨

改法：① 写成 `static int x = 0;`（或编译期常量）；② `malloc(n * sizeof(int))` 并检查 NULL；③ 删掉 p 或真正用它并在结束前 free；④ 结尾 `free(arr); free(p);`。

常见错法：把「没 free」当小事——长跑程序（嵌入式/服务端）泄漏到 OOM 是真实事故；-Wall -Wextra 抓不住 ②④，全靠自己养成「malloc/free 成对」习惯。

12-3

思路：static 局部计数器，每次调用 +1 并返回。

```
#include <stdio.h>
int next_frame(void){
    static int n = 0;
    n++;
    return n;
}
int main(void){
    int i;
    for (i = 0; i < 5; i++) printf("帧号:%d ", next_frame());
    printf("\n");
    return 0;
}
```

常见错法：忘了 static 导致每次重置（输出全 1）；把 static 放函数外当全局用（会改变可见范围）。

12-4

思路：malloc 按 n 申请 → 读入 → 求均值 (sum/(double)n) 和最大 → free。

```
#include <stdio.h>
#include <stdlib.h>
int main(void){
    int n, i, *a, max, sum = 0;
    printf("n=? ");
    scanf("%d", &n);
    a = malloc(n * sizeof(int));
    if (a == NULL) return 1;
    printf("%d 个数: ", n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    max = a[0];
    for (i = 0; i < n; i++){ sum += a[i]; if (a[i] > max) max = a[i]; }
    printf("平均 = %.1f  最大 = %d\n", sum / (double)n, max);
    free(a);
    return 0;
}
```

常见错法：sum / n 整数除法；malloc 后不检查 NULL；忘了 free。

思路：malloc(n) 字节 → 循环填 'A' → 打印前 3 个 → free。

```
#include <stdio.h>
#include <stdlib.h>
int main(void){
    int n, i;
    char *buf;
    printf("缓存大小 n=? ");
    scanf("%d", &n);
    buf = malloc(n);
    if (buf == NULL) return 1;
    for (i = 0; i < n; i++) buf[i] = 'A';
    printf("缓冲前 %d 字节: ", n < 3 ? n : 3); /* 护栏: n<3 时只打印 n 个, 绝不越界读 */
    for (i = 0; i < 3 && i < n; i++) printf("%c ", buf[i]);
    printf("\n");
    free(buf);
    return 0;
}
```

常见错法：char 申请不用乘 sizeof (sizeof(char)=1 恒成立, 写了也对); malloc 后忘记检查 NULL 就直接用 buf[0]。

延伸：真实串口缓冲常用「环形队列」, 先掌握 malloc/free 配对, 数据结构课再深入。

12-6 复习题索引：去书上做 Ch12 复习题 1、3、5 (题号对应主教材, 原题不入本册)。